Fashion MNIST CNN design and results

Exercise 1

Data and model structure

Each input is a 28 x 28 grayscale image. I use 54,000 images for training, 6,000 for validation and 10,000 for testing. Validation helps me choose the model weights. The test set measures performance on images not used to update the weights.

I divide pixel values by 255 to put them between 0 and 1. I one-hot encode each label into ten values and add one grayscale channel before the first convolution. Seed 42 makes the data split repeatable.

Stage	Main settings	Output per image
Convolution 1	32 filters, 3 x 3, stride 1, GELU	28 x 28 x 32
Max pooling 1	2 x 2, stride 2	14 x 14 x 32
Convolution 2	64 filters, 3 x 3, stride 1, GELU	14 x 14 x 64
Max pooling 2	2 x 2, stride 2	7 x 7 x 64
Convolution 3	64 filters, 3 x 3, stride 1, GELU	7 x 7 x 64
Classifier	Flatten, Dense 128, GELU, dropout 0.3	128 values
Output	Dense 10, softmax	10 probabilities

All three convolutions use one-pixel replicate padding.

Why use these convolutions

Three layers and 3 x 3 kernels. I want to learn local patterns first and then combine them in deeper layers. A 3 x 3 kernel examines a position and its immediate neighbors. Deeper layers combine information from a larger area of the original image.

Stride 1. The filter moves one position at a time. I avoid reducing the image size during convolution and leave spatial reduction to pooling. This does not mean that every detail is preserved.

Why chose the remaining values

32 then 64 then 64 filters. The first layer works on larger 28 x 28 maps. Using 32 filters instead of 64 there reduces computation and memory, with the aim of saving time. After pooling, the maps are smaller: 14 x 14 and 7 x 7. I use 64 filters in those layers to learn more feature combinations. Actual speed also depends on the number of input channels.

Replicate padding. I copy the nearest edge value into a one-pixel border. This lets the filter examine neighborhoods centered on the image boundary without inserting a zero border. It keeps the spatial size after convolution. The copied values are an assumption about the boundary, not new image information.

GELU activation. I chose GELU because it is smooth and can pass gradients for negative inputs. ReLU has a zero gradient for negative inputs. GELU may help learning in this region, but it does not guarantee better results. ReLU also supports backpropagation, and GELU gradients can still become very small.

2 x 2 max pooling. I keep the largest response in each small neighborhood. Pooling with stride 2 reduces 28 to 14 and then 14 to 7. I stop pooling at 7 x 7 to keep more spatial detail before classification. Max pooling keeps the strongest response, which is moslty the most useful information.

Dense 128 and dropout 0.3. The dense layer combines the 3,136 flattened features. I retained 128 hidden units as a starting size and used GELU here too. Dropout switches off 30% of these activations during training in order to transfer a part of the information of the dropout switches to the rest. This help us when some value missing from a test to not loss it information

Ten softmax outputs. There are ten clothing classes, so the final layer produces ten probabilities. I use categorical cross-entropy because the labels are one-hot encoded. Lower loss means that the model assigns more probability to the correct answers.

Training choices

Batch size 128. Assignment 1 used Iris, which has only 150 examples in total. Here I train on 54,000 images. Increasing the batch size from 32 to 128 reduces weight updates from 1,688 to 422 per epoch. Each update averages more examples, aiming for less noisy gradients and less update overhead. Smoother curves and better accuracy are not guaranteed.

Forty epochs. I kept the epoch count from Assignment 1. This lets us see both the early improvement and what happens when training continues for longer. The full run uses all 40 epochs.

Adam with learning rate 0.001. I retained these as starting training settings. Adam adjusts updates using information from earlier gradients. I did not compare different optimizers or learning rates as the exercise requested.

The model with the lowest validation loss is kept in memory. Test values are recorded after every epoch for the graphs, but they do not update the weights or choose the best model. Seed 42 is also used for training repeatability.

Learning curves and overfitting

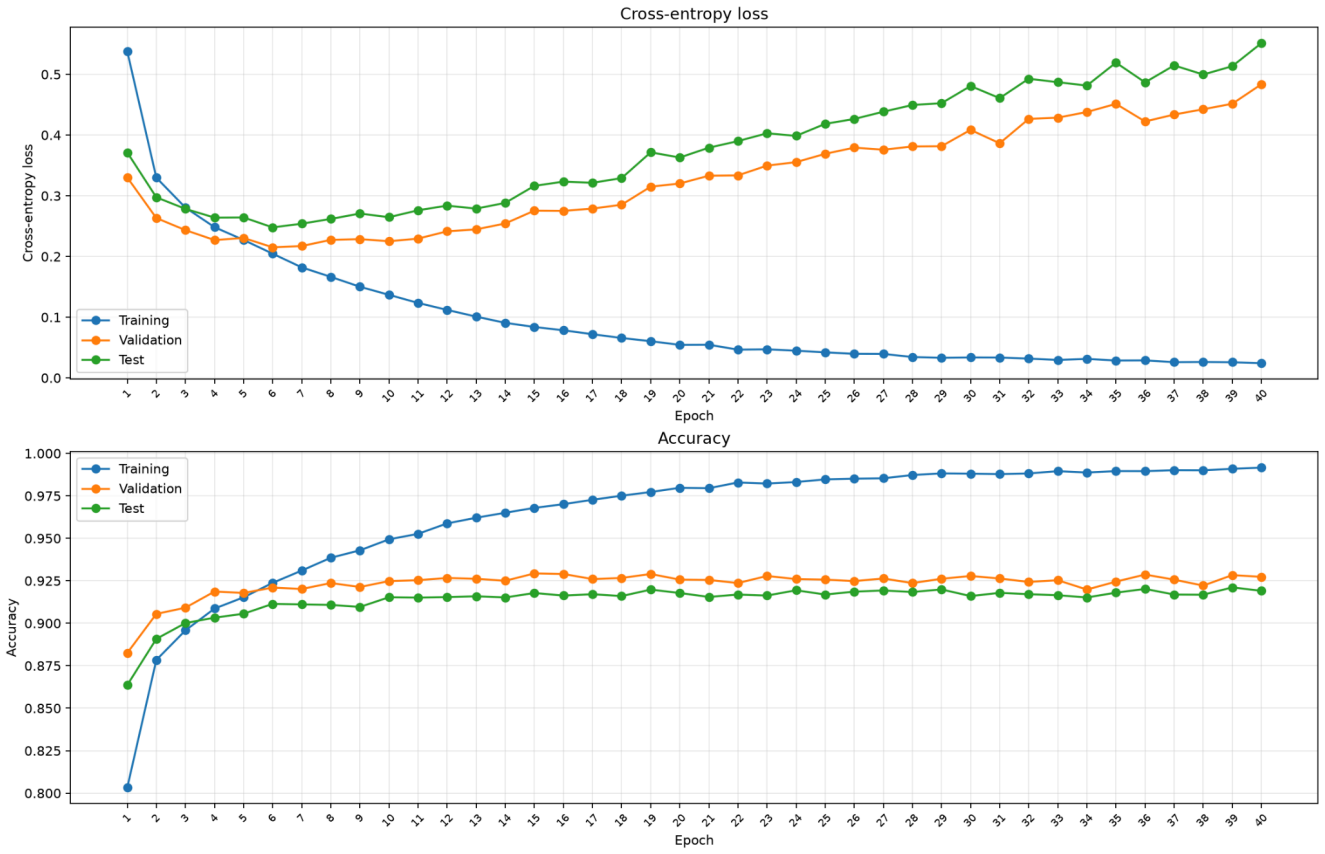


Figure 1. Loss and accuracy for training, validation and test data across all 40 epochs.

During roughly the first six epochs, all three losses fall. This means that the model improves on both the training data and unseen images. At epoch 10, validation loss is 0.2251 and test loss is 0.2647. Their lowest values occur at epoch 6: 0.2151 validation loss and 0.2479 test loss. I use epoch 10 in the discussion because it has better accuracy with a similar validation loss.

After this point, training loss continues to fall, but validation and test loss generally rise, with some short fluctuations. By epoch 40, training loss is 0.0242, validation loss is 0.4837, and test loss is 0.5514. The model fits the training examples more closely while its probability predictions on unseen images get worse. This is evidence of overfitting from around epoch 10.

Why accuracy does not show this as clearly

Accuracy only checks whether the class with the highest probability is correct. Cross-entropy also checks how much probability the model gives the correct class. For one image, the loss is minus the natural logarithm of that probability.

For example, suppose a shirt is wrongly predicted as a coat. If the probability of the correct shirt class falls from 0.20 to 0.02, both predictions are still wrong, so accuracy does not change. However, the loss for the 0.2 is much smaller than the loss for 0.02 probability. A few very confident mistakes can raise the average loss while accuracy stays similar.

Validation accuracy stays around 92 to 93 percent and test accuracy around 91 to 92 percent for much of the run. They do not show the same clear decline as the loss curves. However, training accuracy rises towards 99 percent, so the growing accuracy gap is also a warning sign. Loss gives the clearer signal here.

Test predictions by clothing class

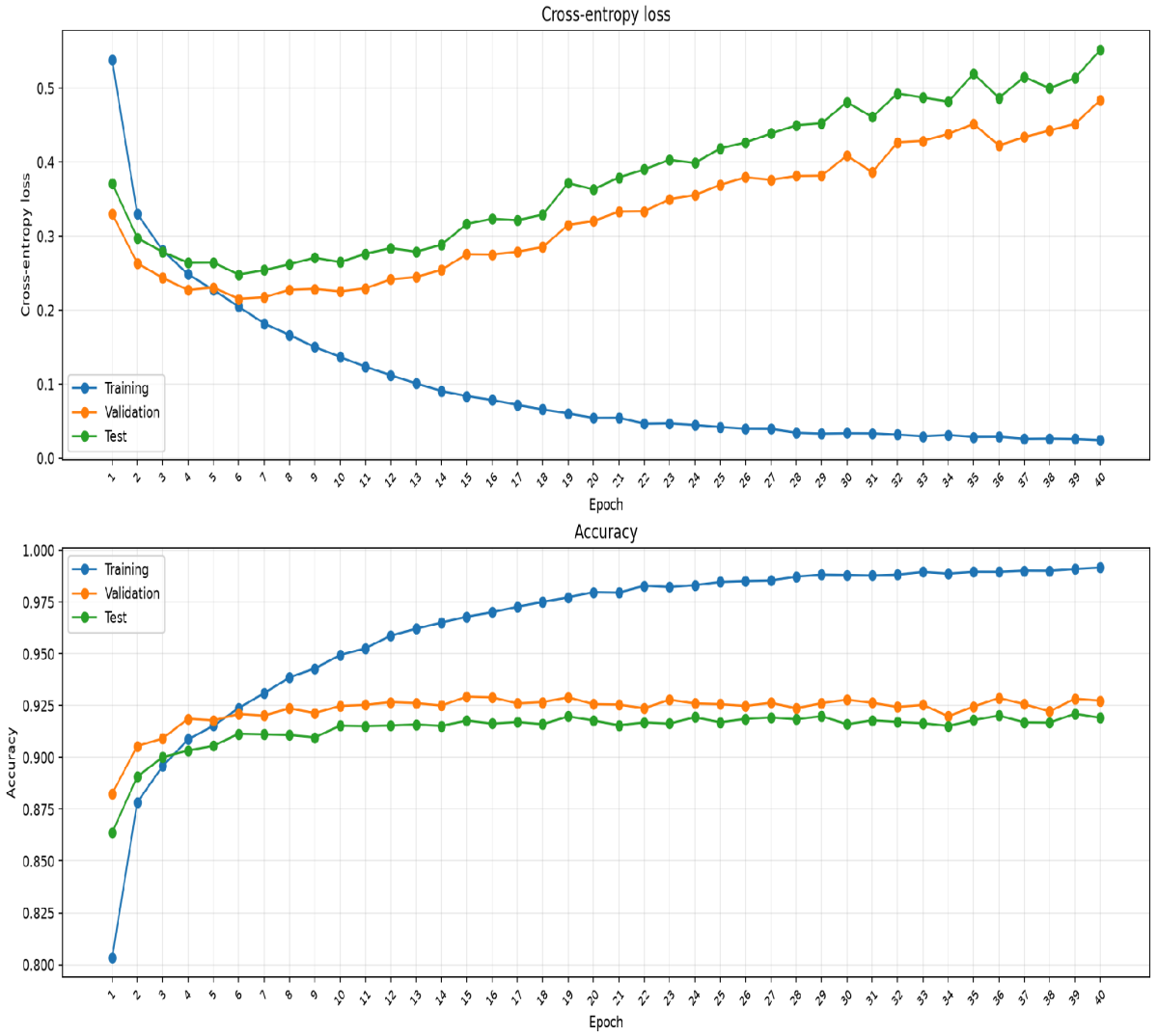
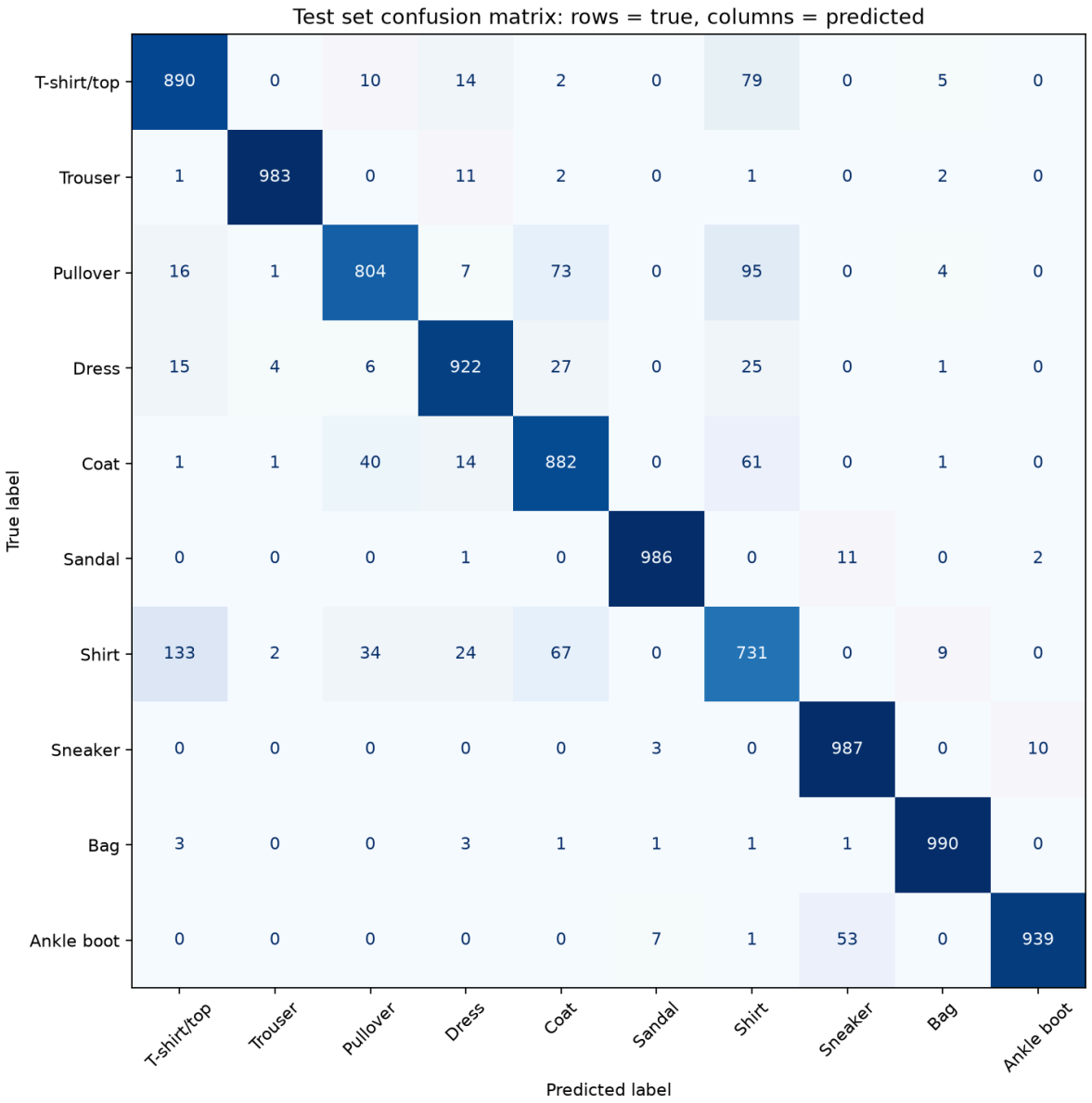


Figure 2. Test confusion matrix for the weights selected using the lowest validation loss. Rows are true classes and columns are predictions.

The diagonal contains correct predictions. Adding its values gives 9,114 correct predictions out of 10,000 test images, or 91.14 percent accuracy. This matrix describes the selected model, not necessarily the model at epoch 40.



Bags, sneakers, sandals and trousers are classified well: 990, 987, 986 and 983 correct predictions out of 1,000 for each class. Shirts are harder, with 731 correct predictions. The model predicts 133 shirts as T-shirts/tops. These errors are easily explained because of the similar clothing shapes.

Exercise 2

Part i Residual connections and gradients

A residual connection sends the input around a convolutional layer and adds it to the convolution output. A residual block calculates output = F(input) + shortcut(input). F is the convolutional path. The shortcut gives gradients a more direct path to earlier layers during backpropagation. This can make training easier, but it does not guarantee better accuracy or prevent overfitting.

Part ii Residual CNN model

The residual model keeps the same CNN architecture from Exercise 1: three 3 x 3 convolutions with 32, 64, and 64 filters; two 2 x 2 max-pooling layers; Dense 128; dropout 0.3; and ten softmax outputs. The only architectural change is one residual shortcut around each convolution.

The first residual block changes 1 channel to 32 channels, so the shortcut keeps its input channel and appends 31 zero channels. The second block changes 32 channels to 64 channels, so its shortcut appends 32 zero

channels. The third block already has 64 channels, so it uses an unchanged shortcut. The convolutions still use 3 x 3 kernels to learn local clothing patterns such as edges, sleeves, shoe shapes, and bag handles.

Residual block	Shortcut adjustment
Block 1	Append 31 zero channels
Block 2	Append 32 zero channels
Block 3	Identity shortcut

The main convolutional path keeps the one-pixel replicate padding from Exercise 1. Zero padding is used only in the shortcut channels so that the shortcut and convolution output have the same number of channels before addition, as requested from the assigment. The shortcuts have no trainable weights.

Part iii Training results and comparison

Training settings. The residual CNN uses the same 40 epochs, batch size 128, Adam learning rate 0.001, data split, seed and the same number of trainable parameters as Exercise 1. Training and per-epoch test evaluation took 971.5 seconds for Exercise 1 and 798.1 seconds for the residual CNN so we can identify an additional time around 21.73%.

Overfitting. Test accuracy does not clearly show overfitting for either model because it remains near the same level after few epochs. Cross-entropy loss gives a clearer signal. For Exercise 1, we identify overfitting after epoch 10: later validation and test loss rise while training loss continues to decrease. We use epoch 10 rather than epoch 6 because it has better accuracy with a similar validation loss. For the residual CNN, this pattern appears after epoch 12.

Comparison with Exercise 1. The two models achieve very similar results. The residual CNN delays the loss-based overfitting pattern by about two epochs, from after epoch 10 to after epoch 12. Across all epochs, it has higher mean and highest validation accuracy. However, the Exercise 1 CNN has slightly higher mean and selected test accuracy and a slightly lower mean test loss. The improvement is therefore a small delay in overfitting, not a clear overall test-accuracy improvement.

Metric	Exercise 1 CNN	Residual CNN
Mean validation accuracy	92.31%	92.41%
Highest validation accuracy	92.93%	93.10%
Mean validation loss	0.3290	0.3236
Lowest validation loss	0.2151	0.2220
Mean test accuracy	91.36%	91.30%
Highest test accuracy	92.11%	91.96%
Mean test loss	0.3793	0.3868
Lowest test loss	0.2479	0.2574
Selected test accuracy	91.14%	90.81%

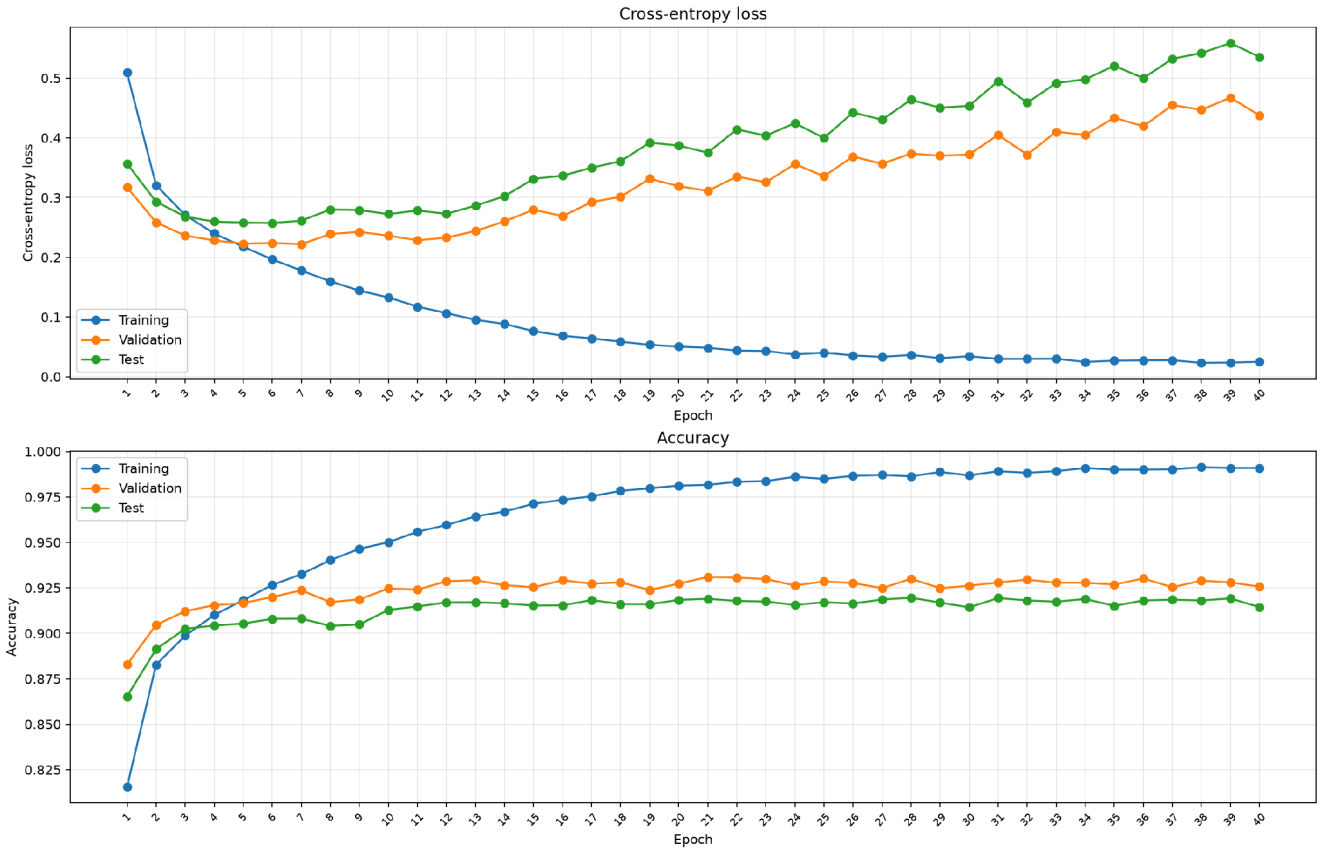


Figure 3. Residual CNN loss and accuracy for training, validation, and test data across all 40 epochs.

Learning curves and overfitting in the residual CNN

Training loss continues to decrease, while validation and test loss are similar at first and then generally rise after epoch 12. Validation and test accuracy remain close to 92 percent and 91 percent, so accuracy does not show the same warning clearly. This is similar to Exercise 1, where loss gives the clearer overfitting signal.

Residual CNN test predictions by clothing class

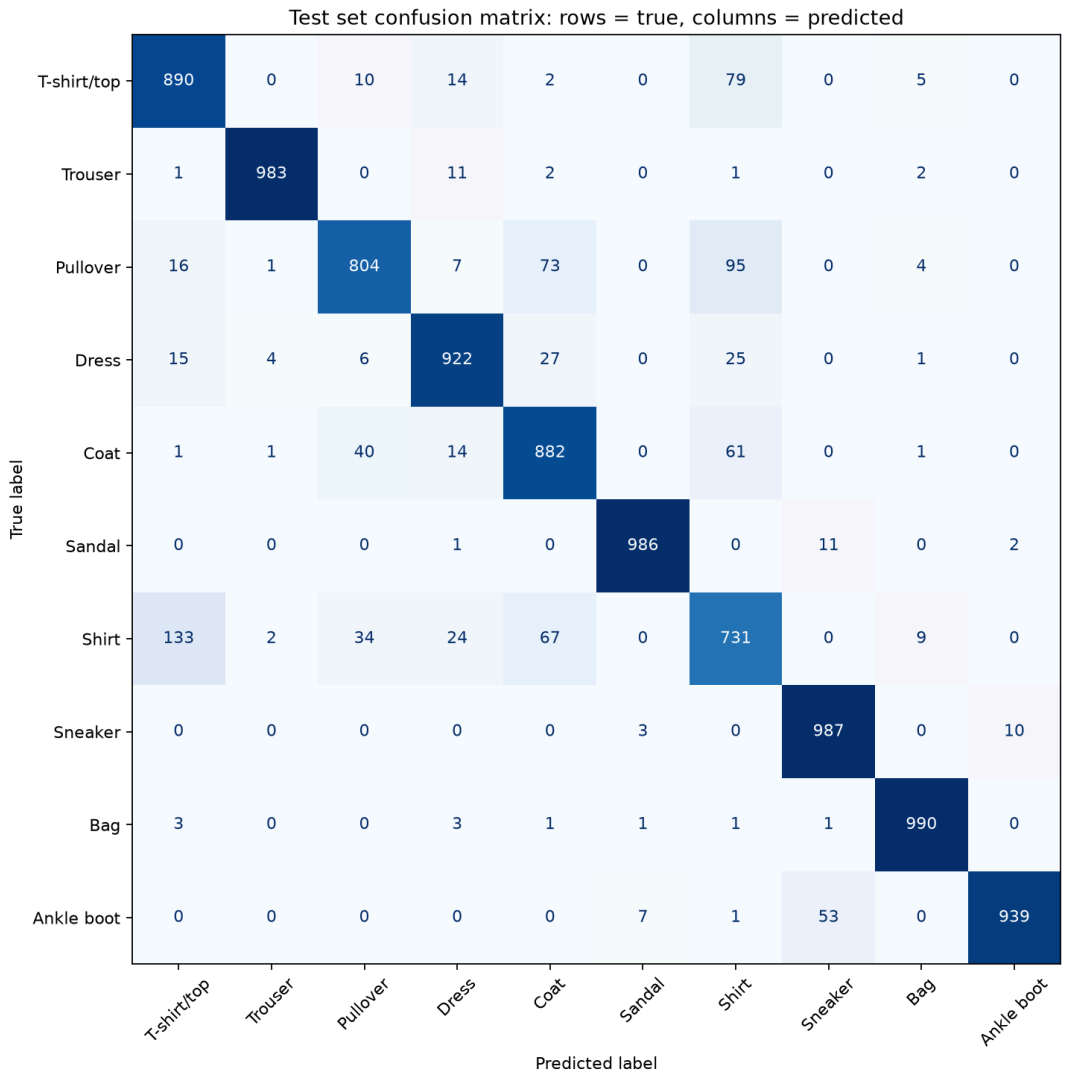


Figure 4. Residual CNN test confusion matrix for the weights selected using the lowest validation loss. Rows are true classes and columns are predictions.

This confusion matrix shows which clothing classes the selected residual CNN classifies correctly and which classes it confuses. Both models confuse shirts with T-shirts/tops, which is expected because these categories have similar shapes. The residual CNN also has difficulty separating pullovers from coats, and shirts. This is similar to the Exercise 1 confusion matrix, so the residual shortcuts do not remove the difficult clothing-class confusions. The shirt result is noticeably better with residual connections: the residual CNN correctly classifies 769 of 1,000 shirts, compared with 731 of 1,000 shirts in Exercise 1. This is 5.18% more correct shirt predictions.